HTTP, State Management, and the Web Application Stack

Introduction to Web Technology — Week 3

This module continues directly from the previous discussion on the internet's architecture, the evolution of the web, and the Domain Name System. Having established *how* a client locates a server, we now turn to *what happens once the connection is made* — the HTTP protocol itself, how the web fakes memory using cookies, how HTTPS keeps that conversation private, and how browsers and servers actually assemble the pages you see.

Index

- GET vs. POST — Asking Nicely vs. Handing Over a Package
- Anatomy of an HTTP Request Message
- Seeing HTTP With Your Own Eyes: netcat and curl
- HTTPS — Wrapping HTTP in a Trench Coat of Encryption
- HTTP's Amnesia Problem, and Why It Matters
- Cookies — The Web's Sticky Notes
- What Cookies Are (Legitimately) Used For
- First-Party vs. Third-Party Cookies — Being Watched Without an Invitation
- Browser Rendering — From Tags to Pixels
- Web Caches — Letting Someone Else Do the Fetching
- Static vs. Dynamic Websites
- Putting It All Together: The Web App Workflow
- Recap
- Think-About-It Questions

GET vs. POST method in HTTP

HTTP communication is built entirely on a **client–server model**: the client (browser) sends an **HTTP request message**, and the server replies with an **HTTP response message** carrying the requested content. Every request message carries several fields, but the very first thing it announces is *what kind of operation* it wants — most commonly **GET** or **POST**.

- **GET** — used to *fetch* data. Clicking a hyperlink, for instance, triggers a GET request for the target page.
- **POST** — used to *send/upload* data to the server. Filling out a form, submitting a comment on a blog, or uploading information all require POST, because data has to travel *to* the server rather than just being read *from* it.

Aspect	GET	POST
Purpose	Fetch/read a resource	Send/upload data
Typical trigger	Clicking a hyperlink	Submitting a form
Data location	Encoded in the URL	Sent in the request body
Example	`GET /somePage.html HTTP/1.1`	`<form method="POST" action="FormProcess.php">`

For example, a form with `method="POST"` and `action="FormProcess.php"` will, the moment the **Submit** button is pressed, package up the form fields (say, *Title* and *Country*) and send them to `FormProcess.php` using HTTP/1.1. A hyperlink, by contrast, simply issues a GET for the linked page and renders whatever comes back.

> **Pro-Tip:** A very common exam trap is assuming GET can safely be used for actions that change server-side data (like placing an order). It shouldn't be — **GET requests should never be used for state-changing actions**, since GET requests are often cached, pre-fetched, or repeated by browsers and proxies without warning.

The HTTP Request Message

An HTTP request message is plain ASCII text — genuinely human-readable. It has three parts:

1. **Request line** — the method (GET/POST/HEAD), the resource path, and the protocol version.
2. **Header lines** — metadata about the client and what it's willing to accept.
3. **A blank line** (`\r\n`) — a carriage-return/line-feed pair that marks the end of the headers.

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Firefox/3.6.10\r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
\r\n
```

The carriage-return and line-feed characters (`\r\n`) terminate every line, and a lone `\r\n` at the very end signals "headers are done — no more fields are coming."

Seeing raw HTTP: netcat and curl

You don't need a browser to talk HTTP — you can do it by hand:

1. **Open a raw TCP connection to port 80** (the default HTTP port) using `netcat` :
   - macOS: `nc -c -v gaia.cs.umass.edu 80`
   - Windows: `ncat -C gaia.cs.umass.edu 80`
2. **Type a GET request manually** and hit Enter twice (the blank line matters — it's what tells the server the request is complete):

   ```
   GET /kurose_ross/interactive/index.php HTTPS/1.1
   Host: gaia.cs.umass.edu
   ```

3. **Read the response** the server sends back — or capture it in Wireshark to see the raw request/response exchange.

Alternatively, `curl` does the same job with far less typing, and conveniently sidesteps some of the manual TLS handling that raw GET-over-port-80 requires for HTTPS:

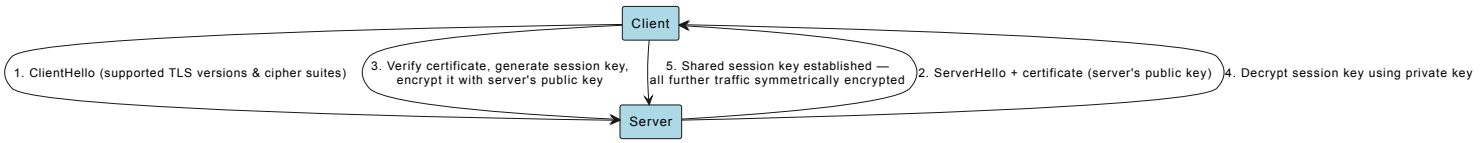
```
curl -L https://gaia.cs.umass.edu/kurose_ross/interactive/index.php
```

Running this returns the page's raw **HTML** — you can see the `<head>` section (containing the page title, e.g. *"Interactive Problems: Computer Networking, A Top-Down Approach"*) and the `<body>` section containing the actual content and hyperlinks. The same underlying HTML can be inspected in any browser's **Developer Tools**: hovering over an element highlights the corresponding line of markup, which is a handy way to learn how a page is structured before HTML/CSS is covered in detail in the next module.

HTTPS Encrypts HTTP with TLS

Plain HTTP has no built-in protection: an attacker sitting "in the middle" could modify the data in transit, steal credentials, or silently reroute traffic through themselves before it ever reaches the real server. **HTTPS** solves this by wrapping HTTP inside the **Transport Layer Security (TLS)** protocol.

TLS relies on **public-key cryptography**: every host (client or server) can hold a *key pair* — a **private key**, kept completely secret, and a **public key**, freely announced to anyone who wants to communicate securely with that host. The public key lets others verify "this is really who they claim to be," while the private key is what lets the host *prove* it.



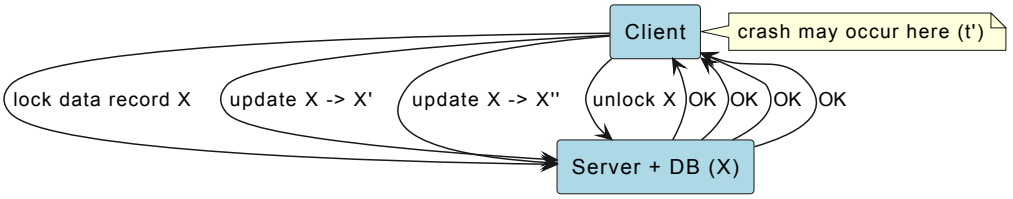
Step by step:

1. The client sends a **ClientHello**, announcing which TLS versions and cipher suites it supports.
2. The server replies with a **ServerHello** plus its **certificate**, which discloses its public key for the first time.
3. The client verifies the certificate, generates a **session key** — a shared secret valid only for this session — and encrypts everything it sends (including the actual GET/POST request) using the server's public key.
4. Because only the server holds the matching private key, only the server can decrypt that message and recover the session key.
5. From this point on, both sides share the same session key, and every message between them is encrypted with it. The underlying transport is still TCP (already reliable), but TLS layers in confidentiality and integrity on top, making HTTPS far more resilient to tampering or eavesdropping.

The Statelessness Problem

HTTP is, by design, a **stateless protocol** — neither the client nor the server is required to remember anything about previous requests. This has advantages (simplicity, no need to track multi-step exchanges, no complicated recovery from a half-finished transaction), but it also creates a real problem.

Consider a server holding a shared variable `X` — an account balance, or the number of seats left on a flight. Suppose the correct behavior requires the client to make **two changes to X, or none at all** (atomicity). The natural way to protect this is locking:



Locking can be **shared** (multiple clients may read, none may write) or **exclusive** (no other client may read or write at all) — the right granularity depends on the application, and will be explored further when database transactions are covered.

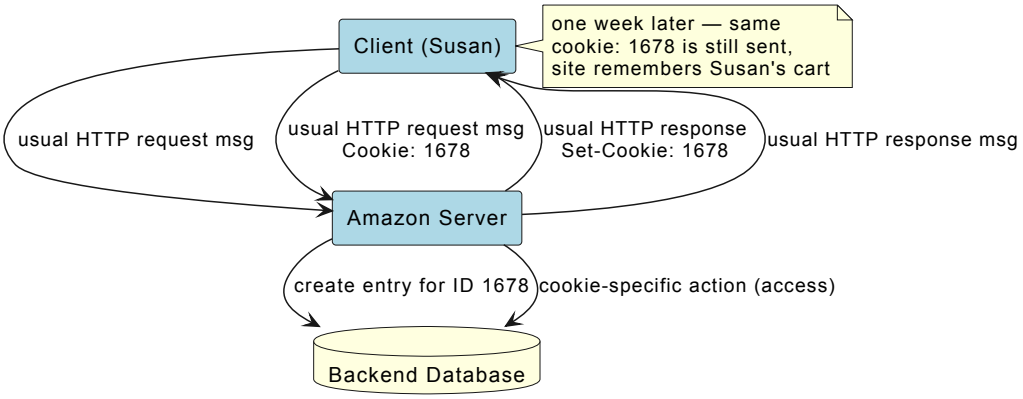
The dangerous scenario: if the **client crashes** right after the first update (at time *t'*) but before unlocking, it wakes up with no memory of having already applied one change — because HTTP itself remembers nothing. If it naively restarts and performs two more updates, `X` ends up updated **three times** instead of the required two, silently corrupting the data. This is exactly the failure mode that statelessness invites, and it's the motivating reason the web needed a way to remember things across otherwise-independent requests — enter cookies.

Maintaining user/server state: cookies

A **cookie** is the mechanism websites and browsers use to fake memory on top of a stateless protocol. It rests on **four components**:

1. A **cookie header line** in the HTTP *response* message (`Set-Cookie: ...`).
2. A **cookie header line** in the next HTTP *request* message (`Cookie: ...`).
3. A **cookie file** kept on the user's host, managed by the browser.
4. A **back-end database** entry at the website.

Worked example — Susan visits an e-commerce site:



On Susan's **very first visit**, no cookie exists yet — it can only be generated after the first successful request/response exchange. When that initial request arrives, the server creates a **unique ID** and an accompanying entry in its backend database, then returns that ID via `Set-Cookie` in the response. The browser stores it, and every subsequent request to that same site includes `Cookie: <ID>` — this is exactly how the site recognizes "this is Susan again," even without a login, and can remember her shopping cart a week later, purely because the same browser sends the same cookie value.

> **Pro-Tip:** Remember that the cookie ID itself carries no meaning to the browser — it's an opaque token. All the actual personalization (cart contents, preferences, browsing history) lives in the **server's database**, keyed by that ID. The cookie is just the lookup key.

HTTP Cookies: Comments

What cookies enable:

- Authorization (recognizing a returning, logged-in user)
- Shopping carts
- Recommendations (based on browsing/purchase history)
- Session state (e.g., staying logged into webmail)

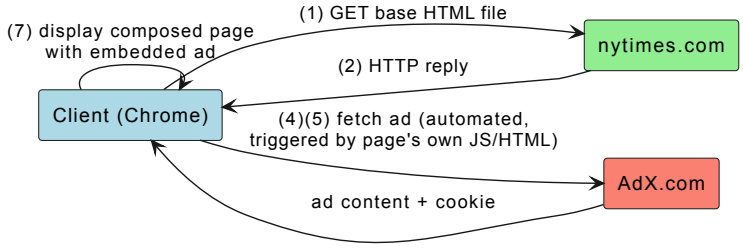
Where state is actually kept, given that HTTP itself has none:

- *At protocol endpoints:* state is maintained at the sender/receiver across multiple transactions.
- *In messages:* the cookie header lines carry that state information along with each HTTP message.

> **Aside — Cookies and privacy:** Cookies let a site learn a great deal about a visitor's activity on *that* site. The real concern is **third-party persistent (tracking) cookies**, which let a single identity be tracked across *many* different websites — something explored in detail next.

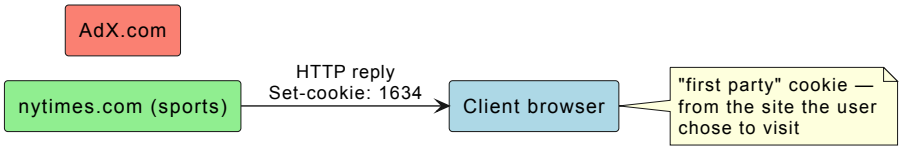
Cookies: Tracking a user's browsing behavior

Consider loading a New York Times page that also embeds an advertisement:

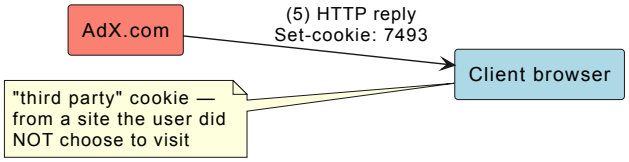


Loading the ad is **not** something the user consciously chose to do — it's triggered automatically by code embedded in the page (JavaScript/HTML placeholders that the browser fills in on the fly). In the process, the browser also exchanges cookie information with the ad network.

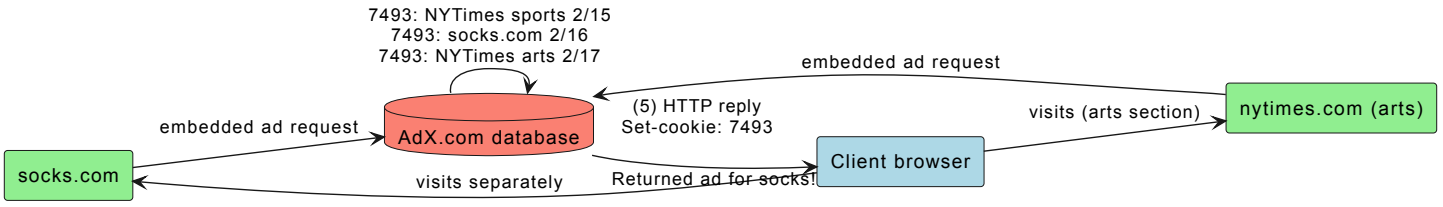
First-party cookie — set by nytimes.com, the site the user *deliberately* visited:



Third-party cookie — set by AdX.com, a site the user *never chose* to visit at all, purely as a side effect of the ad being embedded on nytimes.com:



One day later, the same browser visits an unrelated site, `socks.com`, which *also* happens to use AdX.com for its ads. Because the browser already carries AdX's cookie (`7493`) from the NYT visit, AdX recognizes the same browser and can link the two visits:



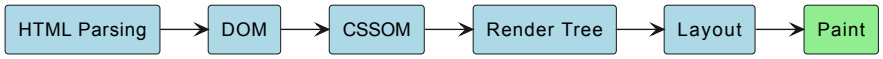
The next time the client visits nytimes.com, AdX — now recognizing cookie `7493` and its accumulated browsing history — serves a **targeted ad for socks**, even though the user never told either site about their sock-shopping interest. This is the essence of cross-site tracking: **AdX can reconstruct a browsing profile spanning multiple unrelated sites**, purely from a cookie the user never explicitly agreed to at either of those sites

Why this matters:

- First-party cookies track behavior *on a single site*.
- Third-party cookies track behavior *across many sites*, without the user ever knowingly visiting the tracker.
- Tracking can be entirely **invisible** — rather than a visible ad, it could just as easily be an invisible tracking pixel/link.
- Because of this, **Firefox and Safari disable third-party cookies by default**, and Chrome moved to disable them as well.
- Once a cookie can identify an individual, it legally counts as **personal data**, bringing it under regulations like **GDPR** — which is why so many sites now prompt for cookie consent.

Browser Rendering

Rendering is the umbrella term for everything a browser does to turn a fetched HTML file into an interactive page on screen: downloading, parsing, laying out, fetching additional assets (images, scripts, video, audio), and finally painting pixels.



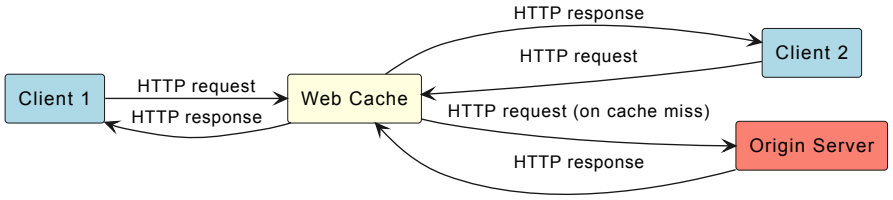
For every additional resource *referenced inside* the HTML — a stylesheet, a script, an image — the browser fires off **separate requests** back to the web server as it parses:

```
1. GET /vacation.html   →  2. HTML returned
3. (parsing begins)
4. GET /chart.js        →  5. JS returned
6. GET /mountain.jpg    →  7. Image returned
8. Once all resources have arrived, the browser lays out and displays the page
```

Static pages, whose content rarely changes, generally have a simpler rendering path. Dynamic pages — where content must reflect the *current* value of something (ticket availability, live prices) — require a considerably more involved rendering and data-fetching process, since correctness depends on showing consistently up-to-date information across many simultaneous visitors.

Web Cache

Goal: satisfy client requests *without* involving the origin server every single time.



The user configures their browser to point at a **local web cache** (also called a **proxy server**), and every request routes through it first:

- **If the object is already in the cache** → the cache returns it directly to the client.
- **Otherwise** → the cache fetches the object from the origin server, stores a copy, *then* returns it to the client.

A web cache is unusual in that it plays **both roles at once**: it's a *server* to the original requesting client, and simultaneously a *client* to the origin server.

Why bother caching?

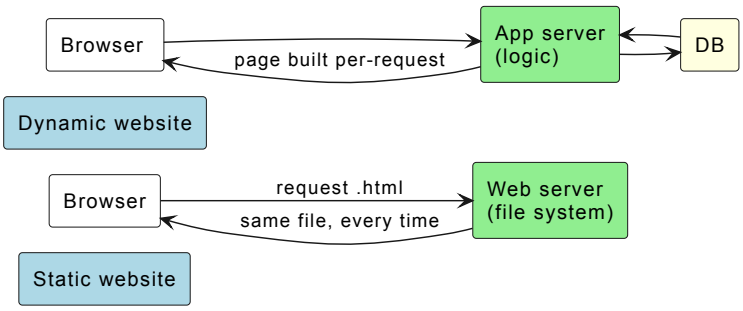
- **Reduces response time** — the cache is physically/network-wise closer to the client than the origin server.
- **Reduces traffic on an institution's access link** — fewer requests have to leave the local network entirely.
- **The internet is dense with caches**, which in practice lets even smaller, less resource-rich content providers deliver content effectively, since intermediate caches absorb repeat load.

The server controls caching behavior via response headers:

```
Cache-Control: max-age=<seconds>
Cache-Control: no-cache
```

> **Pro-Tip:** Caching is a performance win for static content but a genuine liability for dynamic content — a cached page can go **stale**, silently serving outdated ticket prices or seat availability unless the server explicitly limits how long it may be cached (`max-age`) or forbids caching altogether (`no-cache`).

Static Sites Serve Files; Dynamic Sites Build Pages



- **Static:** the *same* HTML file is returned to every visitor — fast, simple, no server-side logic involved.
- **Dynamic:** the server runs code (frequently querying a database) to build a **custom response for every single request**.

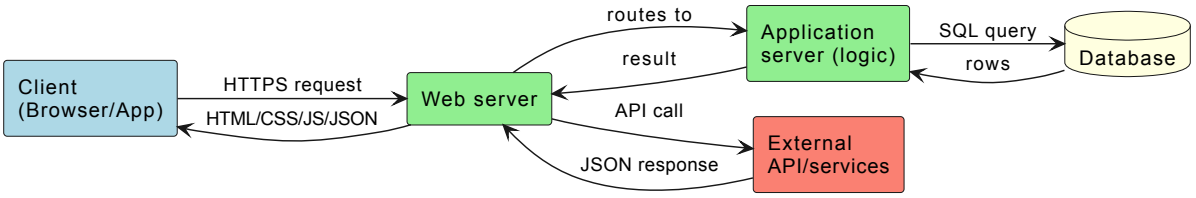
The same conceptual page can be produced either way:

```
Static — about.html (fixed file)
<html>
  <body>
    <h1>About Us</h1>
    <p>Founded in 2020.</p>
  </body>
</html>
```

```
Dynamic — about.ejs (server-rendered)
 <html>
   <body>
     <h1><%= company.name %></h1>
      <p>Founded in <%= company.year %>.</p>
      <p>Visitor #<%= visitCount %></p>
    </body>
   </html>
```

Booking a flight is a good real-world example of why dynamic pages exist at all: prices and seat availability genuinely differ visit to visit, because many clients are competing for the same underlying data, and the server must keep that data accurate and current in real time — not just render a fixed template.

Putting It Together: The Web App Workflow



Typical flow: the browser sends an HTTPS request → the web server routes it to application logic → that logic queries a database and/or calls external APIs → the server assembles a response → the browser renders it.

A static, simple website design rarely needs any of this — but a real application with a back end, multiple concurrent clients, and continuously changing data needs application-server logic that talks to a database *and* potentially calls out to other services/APIs to fetch what it needs.

Popular application stacks:

Stack	OS	Web Server	Database	Scripting/Framework
LAMP	Linux	Apache	MySQL	PHP
MERN	—	Node.js (server & runtime)	MongoDB	Express + React (JavaScript)
WISA	Windows	IIS	SQL Server	ASP.NET
JAM	—	—	—	JavaScript, APIs, Markup

The choice of stack depends heavily on developer preference and application requirements — MERN, for instance, pairs a graph-oriented NoSQL database (MongoDB) with a JavaScript-everywhere approach (React front end, Node.js/Express back end), while LAMP and WISA represent more traditional relational, server-rendered approaches on Linux and Windows respectively.

Recap

- **Internet** = the global network; **Web** = the content layer running on top of it, primarily via HTTP.
- The web evolved: static **Web 1.0** → interactive **Web 2.0** → API-driven, cloud-native **Web 3.0**.
- Client (browser) requests, server responds — the **client–server model**.
- Browsers render pages through a **parse** → **layout** → **paint** pipeline.
- **URLs** identify resources; **DNS** resolves domain names to IP addresses.
- **HTTP** carries requests/responses (GET/POST); **HTTPS** adds **TLS encryption** on top.
- **Cookies** fake statefulness on top of a stateless protocol — useful for carts and logins, but a privacy concern when used by third parties to track users across sites.
- **Web caches/proxies** reduce latency and bandwidth by serving repeat requests locally.
- **Static sites** serve fixed files; **dynamic sites** build a page per request using app-server logic and a database.

Think-About-It Questions

1. HTTP is deliberately stateless, yet the lecture's locking example (`X → X' → X"`) shows real harm when a client crashes mid-transaction. If cookies solve "who is this client," do they, by themselves, also solve the atomicity problem from that locking example? Why or why not — and what additional mechanism would actually be needed?

2. Third-party tracking cookies rely on the *same* ad network being embedded on multiple unrelated sites. If a browser blocked all third-party cookies by default (as Safari and Firefox do), could a site like AdX still track a user across nytimes.com and socks.com using some other technique? Think beyond cookies specifically.

3. A web cache sitting between the client and origin server is described as acting as "both client and server." Suppose the cached page is for a flight-booking dynamic site. Walk through what could go wrong if the cache serves a stale `Cache-Control: max-age=3600` response for seat availability — and propose how the server-side headers could be designed differently to avoid it.

4. During the TLS handshake, the server's public key is sent to the client *before* any encryption is in place. If an attacker on the network path could swap the server's real certificate for their own (with their own public key) during that exchange, what would break — and at which exact step of the handshake would the client actually have a chance to detect the substitution?

5. The lecture contrasts a LAMP-style stack (Apache + MySQL + PHP) with a MERN-style stack (Node.js + MongoDB + React). Beyond "personal preference," what property of an application's *data* (structure, consistency needs, read/write patterns) might make a relational database (as in LAMP/WISA) a better fit than a document/graph database (as in MERN) — or vice versa?